

Agentic Systems: Theory & Practice

**Agentic Systems:
Theory, Architecture, and
A Case Study in Automated Lecture Production**

Technical Documentation
Lecture Production Pipeline v2

Course: Algorithm Analysis
Agentic_lecture_01 System

Generated with Claude Code — Anthropic

Introduction

This document provides a comprehensive technical account of *agentic systems*: software architectures in which one or more AI-driven agents perceive an environment, reason about a goal, select and execute actions, and adapt to feedback—operating with a degree of autonomy that distinguishes them from conventional request-response pipelines.

Part I (Sections 2–3) covers the general theory. Part II (Sections 4–6) describes the specific automated lecture-production system built for this course, analysing how the abstract concepts manifest in concrete engineering decisions.

Part I: Theory of Agentic Systems

What Is an Agent?

The term *agent* originates in philosophy of mind and economics; in computing it was formalised by Russell & Norvig (1995) as:

Definition (Rational Agent). An agent is any entity that *perceives* its environment through sensors and *acts* upon that environment through actuators, with the goal of maximising some performance measure over time.

In the context of large language model (LLM) systems, this definition extends to:

- **Perception:** text, images, tool results, memory contents, conversation history
- **Reasoning:** chain-of-thought generation, planning, self-reflection
- **Action:** function/tool calls, code execution, file system operations, API requests, spawning sub-agents
- **Adaptation:** adjusting strategy based on observation of action outcomes

The Perception–Reasoning–Action Loop

The core operational model of every agent is a loop:

The core Perceive–Reason–Act loop. Memory feeds context into perception; actions produce observations that re-enter the loop.

Unlike a simple function call, the loop *repeats*: the outcome of each action (tool result, error, partial output) is fed back as a new observation, enabling the agent to react, correct course, or pursue multi-step strategies.

Agent Architectures

Several architectural patterns have emerged:

ReAct (Reason + Act)

ReAct interleaves chain-of-thought reasoning with action steps in a single context window:

```
Thought: I need to find which section uses the iterative tab.  
Action: read_file("sections.json")  
Observation: [..., {"id": "15_linear_search", "tab":  
"iterative"}, ...]  
Thought: Section 15 targets iterative tab. I should switch the  
tab first.  
Action: click_tab("iterative")  
Observation: {"btnActive": true, "panelVisible": true}
```

The model decides *what* to think and *what* to do in the same generation step. This is simple but vulnerable to long-horizon forgetting and accumulating errors in the context.

Plan-and-Execute

A *planner* agent decomposes the goal into a sequence of steps; separate *executor* agents carry out each step. The planner may revise the plan based on executor feedback.

Plan-and-Execute: a planner decomposes goals; executors report back.

Reflexion / Self-Critique

The agent generates an output, then a second (or the same) LLM call evaluates it and produces a *reflection*. The reflection is stored in memory and used to improve the next attempt. This directly models the quality-assurance loop seen in our system.

Tool-Augmented Agents

Modern LLMs can invoke structured tools through function-calling APIs. The model emits a JSON payload; a host process executes the corresponding function and returns the result. This pattern is the foundation of our pipeline agents.

Tool-augmented LLM: the model selects tools; a host process executes them.

Memory Systems

Agents typically require multiple layers of memory:

Memory layers in agentic systems.

Type	Scope	Storage	Example
In-context	Current episode	LLM context window	Conversation history
External/Episodic	Cross-session	Files, DB	Memory markdown files
Procedural	Permanent	System prompt / code	Agent behaviour rules
Semantic	Domain facts	Vector DB / docs	Course content
Working	Computation	Variable state	Run directory, counters

In our pipeline, *external episodic memory* is implemented as Markdown files in `.claude/projects/.../memory/`, while *working memory* is the `state.json` file persisted in each run directory.

Agent Evaluation and Quality Assurance

A fundamental challenge in agentic systems is verifying that actions produced the intended outcome. Three strategies exist:

1. **Deterministic verification** — check ground-truth state directly (e.g. DOM inspection, file existence, API status code). Fast and exact but requires programmatic access to state.
2. **LLM-as-judge** — pass the output to a second LLM call that evaluates it against a rubric. Flexible but probabilistic; the judge can err or be inconsistent across calls on identical input.
3. **Human-in-the-loop** — a person reviews and approves the output before the pipeline continues. Slowest but highest fidelity.

Our system uses all three. DOM inspection is ground truth for tab correctness (deterministic); a vision-capable LLM judges screenshot quality (LLM-as-judge); an optional `--no-human-review` flag can bypass human checkpoints.

Failure Modes in Agentic Systems

Common agentic failure modes (left) and mitigations used in this system (right).

Multi-Agent Orchestration

Orchestrator–Worker Pattern

The most common multi-agent topology is hierarchical: an *orchestrator* maintains the high-level goal and delegates subtasks to specialised *workers*.

Orchestrator–worker topology used in the lecture production pipeline.

Inter-Agent Communication Channels

Agents in our system communicate through three channels:

1. **Direct function call** — synchronous; the orchestrator calls a worker function and awaits its return value. Used for TTS, recording, QA.
2. **Shared file state** — `state.json` and the run directory act as a blackboard. Any agent can read or write; the orchestrator coordinates access. Enables crash recovery.

3. **LLM context injection** — worker prompts are constructed by the orchestrator, embedding relevant state (section config, previous QA scores, tab state) before calling the LLM API.

Parallelism and Sequential Constraints

Temporal dependencies in the pipeline. TTS for section $N + 1$ can overlap with recording of N ; assembly is strictly after all sections.

Part II: The Lecture Production Pipeline

System Overview and Motivation

The system automates the production of narrated, screen-recorded lecture videos from an interactive HTML educational tool. The goal is to replace the hours of manual effort required to record, narrate, and edit a lecture with a fully agentic pipeline that: (a) generates narration scripts, (b) synthesises speech, (c) controls the browser tool to show the right content at the right time, (d) captures synchronized video, (e) performs quality assurance at each step, and (f) assembles the final video.

High-level pipeline stages.

Input: Interactive HTML Tool

The tool is a single-page application (Asymptotic_Notation_Explorer.html) implementing five tabs:

The five tabs of the interactive HTML tool.

Tab	data-mod	Content
Notation Guide	notations	Big-O/ Ω / Θ definitions, comparison charts
Iterative Analysis	iterative	Step counters, complexity charts per sort
Recursive Analysis	recursive	Recursion trees, merge/quick/binary search
Proof Builder	prover	Interactive formal proof construction
Challenge Mode	challenge	Practice problems with feedback

A critical engineering issue was discovered: space-utilisation fixes had added `display: flex` as an *inline style* to all tab panels. Because inline styles have higher CSS specificity than class rules, the class `.module{display:none}`

was overridden, making all panels simultaneously visible. The fix was to explicitly set `display:none` in the inline style of inactive panels.

Stage 1: Tool Quality Assurance

Before any recording, a vision-capable LLM inspects a screenshot of the loaded tool and evaluates it against a structured rubric (viewport overflow, interactive elements, tab accessibility). If the score falls below 75/100, the tool-fixer agent is invoked to patch the HTML.

Stage 3: Per-Section Recording Loop

This is the core of the pipeline. For each of the 45 sections:

Detailed flow for each section in Stage 3.

Tab Switching: Engineering vs Prompt-Based Verification

The tab-switching problem illustrates a fundamental principle: *prompt-based verification cannot replace engineering verification*.

Earlier versions asked the QA LLM to confirm the correct tab was showing. The LLM produced false positives (the Notation Guide header also appears as a panel title, confusing the vision model). The engineering solution is a DOM query executed in the page JavaScript context:

```
const state = await page.evaluate((tab) => {
  const btn = document.querySelector(`.tab[data-
    mod="${tab}"]`);
  const panel = document.querySelector(`#viz-${tab}`);
  return {
    btnActive: btn?.classList.contains('active') ?? false,
    panelVisible: window.getComputedStyle(panel).display !==
      'none'
  };
}, expectedTab);
// Both must be true before recording starts
if (!state.btnActive || !state.panelVisible) { /* retry click
  */ }
```

This is deterministic: it queries the live render tree, not a screenshot interpretation.

GPU-Accelerated Rendering

The original implementation used Playwright's default `chrome-headless-shell` binary. This binary forces `--disable-gpu` internally: CSS display changes applied in the DOM were not flushed to the compositor before screenshots were taken, causing stale frames to be captured despite correct DOM state.

The fix uses `channel: 'chrome'` with `headless: true`, which invokes the real Chrome binary in "new headless" mode (introduced Chrome 112). This engages the full GPU compositing pipeline (Metal on Apple Silicon):

Old (headless-shell, software rendering) vs new (real Chrome, Metal GPU) rendering pipeline. The GPU path flushes composited frames before screenshot capture.

HTTP Server vs file:// URL

Serving the tool over a local HTTP server (Node.js built-in `http` module, random port) resolves several `file://` URL limitations:

- `waitUntil: 'networkidle'` is meaningful (real HTTP requests exist)
- `Cache-Control: no-store` ensures each page load is fresh
- Query-string parameters and reload semantics behave as expected

```
function startToolServer(toolPath) {
  const server = createServer(async (req, res) => {
    const filePath = resolveFilePath(req.url, toolPath);
    res.writeHead(200, {
      'Content-Type': mimeType(filePath),
      'Cache-Control': 'no-store' // always fresh
    });
    createReadStream(filePath).pipe(res);
  });
  server.listen(0, '127.0.0.1', () => { // port 0 = OS
    assigns free port
    resolveP({ server, url:
      `http://127.0.0.1:${server.address().port}/` });
  });
}
```

Stage 3c: Quality Assurance Agent

After each section is recorded, a vision-capable LLM receives a sample frame and evaluates a structured checklist:

QA checklist items, verification method, and scoring impact.

Check	Type	Hard fail?	Score cap
correct_tab	DOM (ground truth override)	Yes	0 if false
text_readable	LLM vision	No	Warning only
pointer_visible	LLM vision	No	-10 pts
content_matches	LLM vision	Yes	0 if false

Check	Type	Hard fail?	Score cap
right_panel_active	LLM vision	No	≤ 50 if false
space_utilized	LLM vision	No	-15 pts

A key design decision is that `correct_tab` is *overridden* with the DOM-verified result from `page.evaluate()`, regardless of what the LLM judge reports. This prevents vision hallucinations from masking a real tab error.

Pitfall: Retry Loops on Unresolvable Failures

An early version made `text_readable` a hard-fail trigger. Chart axis tick labels are rendered at 10px by Chart.js regardless of retries. This caused every chart section to exhaust all retries (recording the same content 3 times), wasting $3 \times 30s \times 30 \text{ sections} \approx 45 \text{ minutes}$ per run. The fix: only retry on failures that are *causal* (wrong tab, content mismatch); style issues become warnings.

Stage 5: Video Assembly

After all sections complete, `assembleFullLecture()` uses FFmpeg to:

1. Convert each section's frame sequence + audio into a per-section `.mp4` segment (H.264, VideoToolbox hardware encoder on Apple Silicon)
2. Concatenate all segments using FFmpeg's `concat` demuxer
3. Optionally burn subtitle bars via `drawtext` filter

Assembly stage: per-section frame sequences are encoded and concatenated.

State Management and Crash Recovery

The pipeline writes a `state.json` file after each section completes. This enables resuming interrupted runs:

```
// After each section
state.data.completedSections.push(sectionId);
state.data.sections[sectionId] = {
  framesDir, audioPath, qaScore, qaChecklist, timestamp
};
await state.save(); // atomic write to state.json
```

On restart, the pipeline skips sections already present in `state.data.completedSections`, making every run *idempotent* with respect to completed work.

Lessons Learned and Design Principles

Principle 1: Engineering Verification Over Prompt Verification

Whenever the correctness of an action can be verified programmatically (DOM state, file existence, HTTP status), do so. LLM-as-judge is powerful but non-deterministic: the same frame can receive different scores across calls. Reserve LLM judgement for inherently subjective evaluations (readability, clarity, pacing).

Principle 2: Identify the Root Cause Layer

The wrong-tab bug existed at three layers simultaneously:

1. HTML: `display:flex` inline style overriding CSS `display:none`
2. Rendering: software compositor not flushing CSS changes before screenshot
3. Verification: LLM judge misidentifying the active tab from visual context

Fixing only one layer left the others as latent failures. The permanent fix addressed all three.

Principle 3: Bound Retry Behaviour

Every retry loop must have:

- A maximum iteration count
- A check that the failure is *resolvable by retrying* (idempotent retries on a deterministically failing condition are pure waste)
- A fallback that logs and continues rather than hanging

Principle 4: Separate Concerns Between Agents

Each agent has one responsibility:

- TTS Agent: text → audio file
- Recorder: browser state + audio → frame sequence
- QA Inspector: frame sample → structured quality report
- Assembler: frames + audio → .mp4 segments → final video

The orchestrator (`pipeline.mjs`) contains the control logic but no domain implementation. This separation makes each component testable and replaceable independently.

Principle 5: Use Infrastructure, Not Workarounds

Serving the HTML tool over a real HTTP server (instead of `file://` URLs with reload hacks) is an example of using the right infrastructure. Each workaround applied to a `file://` URL (cache-bust query strings, forced `page.reload()`, extended timeouts) was a symptom of using the wrong transport. One infrastructure change eliminated all of them.

System Architecture Summary

Full system architecture showing agents, external services, storage, and data flow.

Conclusion

The lecture production pipeline demonstrates that agentic systems can automate complex, multi-step creative and technical workflows when designed with the following principles in mind: clear separation of agent responsibilities, deterministic verification where possible, bounded retry behaviour, proper infrastructure rather than workarounds, and persistent state for crash recovery.

The recurring tab-switching bug—which caused hours of wasted runs—was ultimately a lesson in the difference between *symptoms* and *root causes*. Three separate layers each needed fixing. An agent that only patches the symptom (prompt-level instructions to “switch to the correct tab”) cannot fix a rendering pipeline deficiency. The engineering solution (HTML inline styles + DOM verification + GPU compositor) is permanent and unconditional.

Future extensions to this system could include:

- Per-section video assembly immediately after recording for mid-run human or agent inspection
- Parallel TTS pre-generation for all sections before recording begins

- A scheduling agent to run overnight with remote status notifications
- Extension to other interactive HTML tools beyond the notation explorer

Cost Analysis: Claude API vs. Claude Code

The API Cost Problem

Building the lecture production pipeline with direct calls to the Anthropic API (Claude API) incurs real, per-token costs at every stage. A single full run of the 45-section pipeline exercises the LLM multiple times per section:

LLM calls per section and approximate token counts.

Agent	Model	Calls/section	Tokens/call (est.)
QA Inspector (vision)	claude-sonnet-4-6	1–3	~2 000 input + 400 output
Section Planner	claude-opus-4-6	1	~4 000 input + 1 000 output
Tool Fixer	claude-opus-4-6	0–2	~8 000 input + 2 000 output
Quality Loop	claude-sonnet-4-6	0–5	~3 000 input + 600 output

For a 45-section run with typical retry rates, the cumulative token consumption can reach **400 000–800 000 tokens per run** across input and output. At current Anthropic API pricing (Sonnet: \$3/M input, \$15/M output; Opus: \$15/M input, \$75/M output), a single full run costs roughly **\$5–\$20 USD**. When the pipeline was being developed and many runs were aborted and restarted due to bugs, the total expenditure across all runs reached a significantly higher figure — costs that accumulate invisibly in the background.

Claude Code as a Cost-Free Alternative

Claude Code is Anthropic’s official CLI for Claude, designed for software engineering tasks. Crucially, when operated within a paid Claude subscription (Max plan), Claude Code usage does *not* incur per-token charges — it is covered by the flat subscription fee.

This changes the economics of agentic pipelines entirely:

- **Unlimited orchestration calls** at no marginal cost
- **Persistent session context** across the full pipeline run

- **Built-in tool use:** file read/write, bash execution, web search, and glob/grep are native tools — no custom function-calling scaffolding needed
- **Multi-agent subprocesses** via the Agent tool (spawns specialised subagents for parallel or focused tasks)
- **Memory system:** a file-based persistent memory at `~/.claude/projects/.../memory/` survives across sessions

Reimplementing the Pipeline with Claude Code

The lecture production pipeline described in this document was *entirely orchestrated by Claude Code* running as the primary agent. The operator (a human) provided high-level instructions; Claude Code:

1. Diagnosed and fixed the tab-switching bug (reading HTML, editing inline styles, verifying with Playwright)
2. Implemented the HTTP server (writing Node.js code, adding imports, testing with a quick `node -e` check)
3. Modified the QA pipeline (editing `pipeline.mjs` to remove the wasteful `text_readable` retry loop)
4. Generated this very document (writing LaTeX, compiling to PDF via Chrome, creating the audiobook)
5. Monitored the running pipeline (reading log files, reporting section counts and tab pass rates)

In other words: **the pipeline that produces the lecture video is itself orchestrated by an agentic system (Claude Code) that is also responsible for building and maintaining the pipeline.** This recursive self-improvement loop — an agent that debugs and extends the tools it uses to accomplish its goals — is a defining characteristic of advanced agentic systems.

Two-level orchestration: Claude Code (subscription, zero marginal cost) orchestrates `pipeline.mjs` which in turn orchestrates task-level agents (API calls).

Key insight: The expensive API calls (QA inspection, TTS) happen inside `pipeline.mjs` using the OpenAI and Anthropic APIs. But the *meta-level* decisions — what to build, how to fix bugs, what to monitor — are made by Claude Code at zero additional cost. Restructuring the system to move more logic into Claude Code's orchestration layer and fewer decisions into direct API calls would further reduce costs.

Pipeline Implementation in Detail

Code Architecture

The pipeline is implemented as a set of ES module (.mjs) files orchestrated by pipeline.mjs. The directory structure is:

```
Agentic_lecture_01/
  pipeline.mjs           # Main orchestrator
  sections.json          # 45-section configuration
  Asymptotic_Notation_Explorer.html # Interactive tool
  agents/
    tts-agent.mjs        # OpenAI TTS wrapper + quality
checks
  recorder.mjs           # Playwright screencast capture
  quality-inspector.mjs  # Vision QA via Claude API
  assembler.mjs          # FFmpeg video assembly
  navigator.mjs          # Browser navigation + cursor
overlay
  section-planner.mjs    # LLM-based section script
planning
  tool-qa.mjs            # HTML tool quality validation
  tool-evolver.mjs       # (disabled) LLM-driven HTML
mutation
  video-qa.mjs           # Post-assembly video quality
checks
  conversation-monitor.mjs # Human review integration
utils/
  claude-agent.mjs       # Claude CLI subprocess wrapper
docs/
  agentic_system_theory.tex # This document
  make_audiobook.mjs      # Audiobook generator
runs/
  run_XXXXX/             # One directory per pipeline run
  state.json             # Resumable state
  frames/                # PNG screenshots per section
  audio/                 # MP3 files per section
  segments/              # Per-section .mp4 files
```

The Orchestrator: pipeline.mjs

pipeline.mjs is a single-file Node.js ESM script (600 lines) that drives the entire pipeline sequentially. Its key design characteristics are:

- **No framework dependency** — pure Node.js with npm packages only for Playwright, OpenAI SDK, and ffmpeg bindings
- **Explicit state machine** — a RunState object is loaded or created at startup and saved after every section
- **CLI flags** for skipping stages: --skip-planning, --skip-quality-loop, --skip-tool-qa, --no-human-review

- **Single browser instance** shared across all sections, restarted on crash detection

```
for (const section of sections) {
  if (state.completedSections.includes(section.id)) continue;
  // resume

  // 1. Generate TTS
  const { audioPath, duration } = await
    generateTTS(section.narration, ...);

  // 2. Switch tab + verify DOM
  await switchTabVerified(page, section.targetTab);

  // 3. Pre-select algorithm if needed
  if (section.algoSetup) await
    page.evaluate(section.algoSetup);

  // 4. Record screencast synchronised to audio
  const frames = await recordSectionLive(page, audioPath,
    duration);

  // 5. QA: DOM check overrides LLM vision for tab correctness
  const qa = await inspectQuality(frames[midFrame], section);
  qa.correct_tab = await verifyTabDOM(page,
    section.targetTab);

  // 6. Retry if tab wrong or content mismatch (NOT
    text_readable)
  if (!qa.correct_tab || !qa.content_matches) { retry;
    continue; }

  state.complete(section.id, { frames, audioPath, qa });
  await state.save();
}
```

The Recorder: recorder.mjs

The recorder uses Playwright's `page.screencast()` API to capture frames at 5 fps during real-time audio playback. The key insight is that the recording is *wall-clock synchronised*: the screencast runs for exactly the audio duration, producing a frame sequence that, when encoded at the correct frame rate, has perfect lip-sync with the narration.

```
export async function recordSectionLive(page, audioPath,
  duration) {
  const frames = [];
  const interval = 1000 / FPS;           // 200ms between
    frames
  const totalFrames = Math.ceil(duration * FPS);

  // GPU layout flush before starting
```

```

await page.evaluate(() =>
    document.body.getBoundingClientRect());
await page.waitForTimeout(200);

const startTime = Date.now();
while (frames.length < totalFrames) {
    const screenshot = await page.screenshot({ type: 'png' });
    frames.push(screenshot);
    const elapsed = Date.now() - startTime;
    const nextCapture = frames.length * interval;
    const wait = Math.max(0, nextCapture - elapsed);
    if (wait > 0) await page.waitForTimeout(wait);
}
return frames;
}

```

The QA Inspector: `quality-inspector.mjs`

The QA inspector is a structured LLM call with a vision payload. A sample frame (PNG) is base64-encoded and sent to `claude-sonnet-4-6` with a detailed rubric prompt. The response is a JSON object matching a fixed schema:

```

const prompt = `
You are a quality inspector for lecture video frames.
Evaluate this screenshot against the following 6-point
checklist:
1. correct_tab: Is the "${expectedTab}" tab active?
2. text_readable: Are chart axis labels and body text legible?
3. pointer_visible: Is the red cursor dot visible?
4. content_matches: Does the visual content match
   "${sectionDescription}"?
5. right_panel_active: Does the right panel show a chart or
   visualisation?
6. space_utilized: Does the content fill at least 60% of
   available space?

SCORING: correct_tab=false => score=0. content_matches=false
=> score=0.
Return JSON: { checklist: {...}, score: 0-100, issues: [...] }
`;

```

Why Serving the HTML Over HTTP Solves the `file://` Problems

The Root Problem: How Browsers Treat file:// URLs

When Playwright loads a page via a file:// URL (e.g. file:///Users/arash/.../Asymptotic_Notation_Explorer.html), the browser applies a fundamentally different set of rules than it does for http:// URLs. These differences caused three separate, compounding bugs in the pipeline before the HTTP server was introduced.

Problem 1: waitUntil: 'networkidle' Resolves Instantly

Playwright's page.goto() accepts a waitUntil option that controls when the navigation is considered complete. The 'networkidle' value means: *wait until there have been no network requests for at least 500 ms.*

```
// With file:// – resolves in <10ms (no network = always idle)  
await page.goto('file:///path/to/tool.html', { waitUntil:  
  'networkidle' });  
// ← code continues here immediately, before the page has  
  rendered
```

```
// With http:// – resolves after real network activity settles  
await page.goto('http://127.0.0.1:PORT/tool.html', {  
  waitUntil: 'networkidle' });  
// ← code continues only after all JS/CSS/font requests  
  complete
```

Why this matters: The HTML tool loads Chart.js and executes initialisation JavaScript that sets up the tab panels, renders charts, and applies CSS classes. With file://, Playwright's navigation promise resolves before any of this has happened, because there are no HTTP requests to wait for. The subsequent page.screenshot() captures a partially-rendered or blank page.

With an HTTP server, the browser makes real HTTP requests for the HTML document, any referenced scripts or stylesheets, and fonts. The networkidle condition correctly fires only after all these resources have loaded and the JavaScript has had time to execute.

file:// triggers *networkidle* before JavaScript initialisation completes. HTTP waits until all resource requests finish and the page is fully rendered.

Problem 2: Page Reload Semantics Are Unreliable for file://

Between recording sections, the pipeline reloads the page to reset interactive state (sliders, selected algorithms, chart state). For file:// URLs, page.reload() behaves inconsistently across Chrome versions:

- Some Chrome versions serve the page from the **disk cache** rather than re-reading the file from disk, meaning HTML edits (such as the `display:none` fix applied mid-run) are not picked up
- The `Cache-Control` header cannot be set for `file://` responses — the browser decides caching behaviour unilaterally
- `waitUntil: 'networkidle'` on reload has the same instant-resolve problem as on initial load (Problem 1 above)

The HTTP server solves this by setting `Cache-Control: no-store` on every response, guaranteeing that every reload fetches a fresh copy of the HTML from disk:

```
const server = createServer(async (req, res) => {
  const filePath = resolveFilePath(req.url, toolDir);
  res.writeHead(200, {
    'Content-Type': mimeTypes[ext] || 'text/plain',
    'Cache-Control': 'no-store' // browser must re-fetch
                                every time
  });
  createReadStream(filePath).pipe(res);
});
```

This also means that if the HTML tool is edited mid-run (e.g. to fix a rendering bug), the very next section reload picks up the change with no special handling required.

Problem 3: Origin Restrictions Break JavaScript APIs

The `file://` origin is treated as a *null origin* by browsers. This causes several Web APIs to silently fail or behave differently:

- **Web Workers** cannot be spawned from `file://` pages in Chrome (used by some Chart.js rendering paths)
- **ES module imports** (`import`) are blocked on `file://` in Chrome without `--allow-file-access-from-files`
- **localStorage** and **sessionStorage** are inaccessible from `file://` null-origin contexts in strict mode
- **Fetch API** calls to relative URLs resolve incorrectly, as there is no base URL to resolve against
- **CORS preflight** checks always block cross-origin requests from a null origin, preventing the page from loading remote fonts or icons

Serving over `http://127.0.0.1` gives the page a proper origin (`http://127.0.0.1:{port}`), resolving all of these restrictions.

Problem 4: Query String Parameters Are Silently Ignored

The original pipeline appended a cache-busting query parameter to force reloads:

```
// Old workaround: append ?cachebust=<timestamp> to force
// reload
await page.goto(`file:///path/tool.html?
  cachebust=${Date.now()}`);
// Chrome ignores query strings entirely for file:// URLs
// This had zero effect – the cached version was served anyway
```

Chrome strips query parameters from `file://` URLs before cache lookup, so `?cachebust=1774291347` and `?cachebust=1774291999` resolve to the same cached file. The workaround *appeared* to work in some tests (coinciding with cache eviction) but was fundamentally inert.

With an HTTP server, query parameters are transmitted to the server and can be acted upon (though with `Cache-Control: no-store` they are no longer necessary).

The HTTP Server Implementation

The server is embedded directly in `pipeline.mjs` using Node.js's built-in `http` module — no external dependencies, no separate process:

```
import { createServer } from 'http';
import { createReadStream } from 'fs';
import { stat } from 'fs/promises';
import { join, basename, dirname } from 'path';

function startToolServer(toolPath) {
  const toolDir = dirname(toolPath);
  const toolFile = basename(toolPath);

  return new Promise((resolve, reject) => {
    const server = createServer(async (req, res) => {
      // Strip query string and fragment from URL
      const urlPath = req.url.split('?')[0].split('#')[0];

      // Map / and /toolFile to the HTML file; everything else
      // is a sibling file
      const filePath = (urlPath === '/' || urlPath ===
        `/${toolFile}`)
        ? toolPath
        : join(toolDir, urlPath.replace(/^\/\//, ''));

      const ext = filePath.split('.').pop().toLowerCase();
      const mimeTypes = {
        html: 'text/html', js: 'application/javascript',
        css: 'text/css', json: 'application/json',
      };
    });
  });
}
```

```

    png: 'image/png', jpg: 'image/jpeg', svg:
    'image/svg+xml'
  };

  try {
    await stat(filePath);           // throws if file
    doesn't exist
    res.writeHead(200, {
      'Content-Type': mimeTypes[ext] || 'text/plain',
      'Cache-Control': 'no-store'    // always fresh
    });
    createReadStream(filePath).pipe(res);
  } catch {
    res.writeHead(404); res.end('Not found');
  }
});

// port 0 = OS assigns a free port automatically
server.listen(0, '127.0.0.1', () => {
  const port = server.address().port;
  console.log(` Tool server:
  http://127.0.0.1:${port}/${toolFile}`);
  resolve({ server, port, url:
  `http://127.0.0.1:${port}/${toolFile}` });
});

server.on('error', reject);
});
}

```

Key design decisions:

- **Port 0:** the OS assigns a random available port, avoiding conflicts with other services running on the machine
- **127.0.0.1:** binds to loopback only — not accessible from the network, so no security exposure
- **Streaming response:** `createReadStream().pipe(res)` sends the file in chunks without loading it fully into memory — important for large HTML files
- **Single server for all sections:** started once before Stage 3 and closed once after Stage 5, avoiding the overhead of creating a new server per section

Summary: `file://` vs `http://` for Playwright Automation

Behavioural differences between `file://` and `http://` in Playwright / Chrome automation contexts.

Behaviour	<code>file://</code>	<code>http://</code>
-----------	----------------------	----------------------

Behaviour	file://	http://
waitUntil: 'networkidle'	Resolves instantly (no network)	Resolves after real requests finish
Reload semantics	May serve from disk cache; Cache-Control header ignored	Respects Cache-Control: no-store; always fetches fresh
Query string cache-busting	Silently ignored by Chrome	Works as expected
ES module imports	Blocked (null origin)	Allowed
Web Workers	Blocked in Chrome	Allowed
localStorage	Restricted (null origin)	Works normally
Relative URL resolution	Fragile / broken	Correct base URL
CORS / fetch to subresources	Blocked (null origin)	Works with proper headers

Framework Comparison: Custom Pipeline vs. LangGraph vs. n8n

Overview

The same agentic lecture production pipeline could, in principle, be built with any of three approaches: (1) our custom Node.js implementation, (2) LangGraph (a Python graph-based agent framework), or (3) n8n (a visual no-code/low-code workflow automation platform). Each embodies a different philosophy about how agent logic should be expressed and executed.

Comparison of implementation approaches for the lecture production pipeline.

	Custom (ours)	LangGraph	n8n
Language	JavaScript/Node.js	Python	JSON / JS expressions
Paradigm	Imperative code	Graph of nodes	Visual workflow
State	state.json file	LangGraph State dict	Workflow execution context
LLM access	Direct API calls	LangChain LLMs	HTTP Request nodes
Tool execution	Native JS + npm	LangChain tools	Built-in nodes + HTTP
Branching	if/else code	Conditional edges	If/Switch nodes

	Custom (ours)	LangGraph	n8n
Retry logic	Manual loop	Retry nodes / edges	Error branch + retry node
Parallelism	Promise.all	Parallel branches	Split/merge nodes
Observability	console.log	LangSmith tracing	n8n execution log
Hosting	Local / any server	LangServe / cloud	n8n cloud / self-hosted
Learning curve	High (code)	Medium (Python + graph)	Low (visual)

LangGraph

LangGraph models an agent workflow as a *directed graph* where nodes are Python functions (or LLM calls) and edges carry state between them. Conditional edges implement branching and retry loops.

```

from langgraph.graph import StateGraph, END
from typing import TypedDict

class PipelineState(TypedDict):
    section: dict
    audio_path: str
    frames: list
    qa_result: dict
    attempt: int

graph = StateGraph(PipelineState)

graph.add_node("tts", generate_tts_node)
graph.add_node("switch_tab", switch_tab_node)
graph.add_node("record", record_node)
graph.add_node("qa", qa_inspect_node)

def should_retry(state):
    qa = state["qa_result"]
    if not qa["correct_tab"] or not qa["content_matches"]:
        if state["attempt"] < 2:
            return "record" # retry edge
    return END

graph.add_edge("tts", "switch_tab")
graph.add_edge("switch_tab", "record")
graph.add_edge("record", "qa")
graph.add_conditional_edges("qa", should_retry)

app = graph.compile()

```

What LangGraph adds over our custom implementation:

- **Built-in state checkpointing** via LangGraph's MemorySaver (our system replicates this manually with `state.json`)
- **Visual graph inspection** — the compiled graph can be rendered as a diagram for debugging
- **LangSmith tracing** — every node invocation is automatically traced, logged, and inspectable in the LangSmith UI
- **Human-in-the-loop** via `interrupt_before` / `interrupt_after` node modifiers (we implement this manually with a CLI prompt)
- **Time travel** — replay from any checkpoint for debugging

What our implementation does better:

- **Playwright integration** is natural in Node.js (the Playwright library is JavaScript-native; the Python bindings are secondary)
- **FFmpeg integration** is simpler via `child_process.execSync` than through Python subprocess wrappers
- **No framework overhead** — no dependency on LangChain's abstractions, which add latency and version-incompatibility risks
- **Transparent logic** — every decision is explicit in the code, not hidden inside a framework node

n8n

n8n is a visual workflow automation tool. Workflows are built by connecting nodes in a canvas, with each node performing a specific action (HTTP request, code execution, file operation, LLM call via built-in AI nodes).

n8n equivalent of our pipeline would look like:

1. **Loop Over Items** node iterating over `sections.json`
2. **HTTP Request** node calling OpenAI TTS API
3. **Execute Command** node running a Playwright script (as a shell command, since n8n cannot control a browser natively)
4. **HTTP Request** node calling Claude API with a base64 screenshot
5. **IF** node checking `qa.correct_tab` && `qa.content_matches`
6. **Loop** back to step 3 if check fails (up to 2 retries)
7. **Execute Command** node running FFmpeg for assembly

n8n advantages:

- **Zero coding** for the workflow structure — business logic is visual and immediately understandable by non-engineers
- **Built-in credentials management** for API keys
- **Execution history** — every workflow run is logged with input/output for every node
- **Scheduling, webhooks, triggers** — easy to run on a cron, triggered by a file upload, or fired by a webhook
- **Self-hosted or cloud** with a web UI accessible from any device (addressing the user's remote monitoring need directly)

n8n limitations for this pipeline:

- **No native browser control** — Playwright must still be invoked as a shell command, making the DOM verification and tab switching logic opaque to n8n
- **Limited custom code** — complex logic (the GPU flush, the 5-attempt DOM retry loop) must live in Code nodes as JavaScript, essentially recreating our pipeline inside n8n boxes
- **File handling is awkward** — passing large binary data (PNG frames, MP3 files) between nodes requires temporary files and path management
- **Stateful resumption** requires external storage (a database node) — n8n's built-in state does not persist partial workflow runs

What Is Common to All Three Approaches

Regardless of framework, every implementation of this pipeline must handle the same fundamental concerns:

1. **External API calls** (OpenAI TTS, Anthropic QA) with authentication and error handling
2. **Browser automation** for tab switching and screen capture (Playwright or Selenium in all cases)
3. **State persistence** to support crash recovery and idempotent re-runs (file, database, or LangGraph checkpoint)
4. **Retry logic** with a bounded maximum attempt count
5. **Media processing** via FFmpeg for video assembly
6. **Quality verification** using LLM vision after each recording

The frameworks differ only in *how* these concerns are expressed and *what* infrastructure they provide for observability, scheduling, and human oversight.

Claude Services and Components in Agentic Systems

Anthropic offers several distinct products and APIs that serve different roles in an agentic architecture. Understanding which component to use for which purpose is essential for cost-effective, correct system design.

Anthropic products and their roles in agentic systems.

Component	What it is	Billing	Best for
Claude API	REST API for text/vision inference. Models: Haiku 4.5, Sonnet 4.6, Opus 4.6	Per input+output token	Automated pipelines, production systems
Claude Code	CLI tool; agentic assistant with built-in tools (Bash, file I/O, web search)	Flat subscription (Max plan)	Development, debugging, meta-orchestration
Claude.ai	Web/mobile chat interface	Flat subscription	Manual tasks, document review, ideation
Workspaces / Projects	Persistent conversation context with file uploads	Included in subscription	Long-running collaborative tasks
MCP (Model Context Protocol)	Standard protocol for connecting tools/data sources to Claude	N/A (infrastructure)	Extending Claude Code with external tools
Claude Agent SDK	Python/JS SDK for building multi-agent systems with Claude	API tokens	Programmatic agent orchestration

How Our Pipeline Uses Claude Services

- **Claude API (Sonnet 4.6)** — used by `quality-inspector.mjs` for vision-based QA of recorded frames, and by `tool-qa.mjs` for validating the HTML tool. These are direct REST calls using the `@anthropic-ai/sdk` npm package.
- **Claude Code** — the meta-orchestrator for the entire project. Every code change, bug fix, diagnosis, document creation, and pipeline restart in this project was performed by Claude Code running as an autonomous agent in the terminal.

- **Claude Code’s built-in Agent tool** — used to spawn specialised subagents (e.g., the Explore agent for codebase analysis, the Plan agent for architectural decisions) without incurring additional API costs.
- **Claude Code’s Memory system** — persistent Markdown files at `~/.claude/projects/.../memory/` store lessons learned (e.g., “never use tool-evolver”, “always skip-quality-loop”), ensuring that hard-won debugging insights persist across sessions.

The MCP Ecosystem

The **Model Context Protocol** (MCP) is an open standard that allows Claude Code (and the Claude API) to connect to external data sources and tools through a standardised interface. MCP servers expose *resources* (files, database records, API responses) and *tools* (callable functions) to the Claude model.

For the lecture pipeline, MCP could be used to:

- Connect Claude Code to the Playwright browser via an MCP browser-control server (eliminating the need for `pipeline.mjs` entirely)
- Expose the `runs/` directory as an MCP resource, allowing Claude Code to query completed sections directly without reading `state.json`
- Integrate with n8n via its MCP node, creating a hybrid visual+agentic workflow

LLM Portability: Replacing Claude with GPT-4o, Qwen, or Others

Which Parts Are LLM-Dependent?

The pipeline has exactly two LLM-dependent components:

1. **QA Inspector** (`quality-inspector.mjs`) — requires a *vision-capable* LLM that can analyse a screenshot and return a structured JSON response
2. **Section Planner / Tool QA / Tool Fixer** — requires a *text* LLM with strong code-generation capability (for rewriting HTML or generating narration scripts)

All other components (TTS, Playwright recording, FFmpeg assembly, HTTP server, DOM verification) are **LLM-independent** and would remain identical regardless of which language model is used.

Replacing Claude with GPT-4o (OpenAI)

GPT-4o is the most direct drop-in replacement. It supports vision, function calling, and JSON-mode responses — the exact capabilities our QA inspector requires.

```
// Replace the Anthropic SDK call:
const response = await anthropic.messages.create({
  model: 'claude-sonnet-4-6',
  messages: [{ role: 'user', content: [
    { type: 'image', source: { type: 'base64', data: frameB64
    }},
    { type: 'text', text: qaPrompt }
  ]}],
});

// With the OpenAI SDK call:
const response = await openai.chat.completions.create({
  model: 'gpt-4o',
  messages: [{ role: 'user', content: [
    { type: 'image_url',
      image_url: { url: `data:image/png;base64,${frameB64}`
    }},
    { type: 'text', text: qaPrompt }
  ]}],
  response_format: { type: 'json_object' }
});
```

The prompt itself needs no modification; the JSON output schema is identical. The only change is the SDK call and the Content format.

Replacing Claude with Qwen-VL (Alibaba)

Qwen-VL is a strong open-source vision-language model available via the DashScope API (Alibaba Cloud) or via Ollama for fully local inference. Using Qwen-VL would make the QA inspection **completely free and private**:

```
// Run locally: ollama pull qwen2.5vl:7b
const response = await
  fetch('http://localhost:11434/api/generate', {
    method: 'POST',
    body: JSON.stringify({
      model: 'qwen2.5vl:7b',
      prompt: qaPrompt,
      images: [frameB64], // base64 PNG
      stream: false,
      format: 'json' // Ollama structured output
    })
  });
const { response: text } = await response.json();
const qa = JSON.parse(text);
```

Trade-offs with local models:

- **Inference speed:** a 7B model on an M5 Max GPU runs at ~30–50 tokens/sec — a QA call takes ~10–20s vs ~2–4s with the Claude API
- **Quality:** Qwen2.5-VL-7B is competitive on structured screenshot analysis but may be less consistent on ambiguous cases than Sonnet 4.6
- **Cost:** zero per-token cost after the initial model download
- **Privacy:** all inference stays on the local machine — no lecture content is sent to external servers

Replacing Claude with Gemini (Google)

Gemini 2.0 Flash supports vision and structured JSON output via the Google AI Studio API or Vertex AI. It has a generous free tier (1 500 requests/day) that could cover an entire pipeline run at no cost:

```
import { GoogleGenerativeAI } from '@google/generative-ai';
const genai = new
    GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genai.getGenerativeModel({
    model: 'gemini-2.0-flash',
    generationConfig: { responseType: 'application/json' }
});
const result = await model.generateContent([
    { inlineData: { mimeType: 'image/png', data: frameB64 }},
    qaPrompt
]);
const qa = JSON.parse(result.response.text());
```

What Cannot Be Replaced: Claude Code as Meta-Orchestrator

While the *task-level* LLM (QA inspector, planner) can be swapped freely, the **meta-level orchestrator** (Claude Code) is harder to replace. Claude Code provides:

- A persistent, tool-rich agentic environment with file I/O, Bash, and web access built in — no scaffolding required
- The ability to *read, understand, and modify its own pipeline code* — a form of self-referential capability that general-purpose LLM APIs do not provide out of the box
- A memory system that persists engineering lessons across sessions
- A subscription model that makes the meta-level reasoning free

An equivalent could be built with OpenAI's Assistants API (with Code Interpreter and file-search tools), Google's Gemini with function calling, or a local o1lama model paired with a custom agent loop. However, each would require building the scaffolding that Claude Code provides natively.

The bottom layer (TTS, Playwright, FFmpeg) is identical across all LLM choices. Only the top two layers change.

9 S. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023.

S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Prentice Hall, 1995.

N. Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS 2023.

L. Wang et al., *Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning*, ACL 2023.

Anthropic, *Claude Technical Report*, 2024.